

Hochschule für Angewandte Wissenschaften Hamburg

Fakultät Technik und Informatik

Department Informatik

Zero-Knowledge-Netzwerktunnel für zensurresistente Kommunikation

Entwurf, Implementierung und emulationsbasierte Evaluation

Bachelorarbeit

Vorgelegt von: Vorname Nachname
Matrikelnummer: 0000000

Erstgutachter: Prof. Dr. Erstgutachter
Zweitgutachter: Prof. Dr. Zweitgutachter

Abgabedatum: TT.MM.JJJJ

Eidesstattliche Erklärung

Hiermit versichere ich an Eides statt, dass ich die vorliegende Bachelorarbeit mit dem Titel “Zero-Knowledge-Netzwerktunnel für zensurresistente Kommunikation” selbstständig und ohne fremde Hilfe verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Die Stellen der Arbeit, die dem Wortlaut oder dem Sinn nach anderen Werken entnommen sind, wurden unter Angabe der Quelle kenntlich gemacht. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Hamburg, den TT. MM. JJJJ

Vorname Nachname

Kurzfassung

Diese Arbeit entwirft und implementiert einen *Zero-Knowledge-Netzwerktunnel* für zensurresistente Kommunikation: ein providerblindes Relay, das nur Chifretext weiterleitet, während die Ende-zu-Ende-Verschlüsselung zwischen Client und Ursprung über das Noise-Protokoll erfolgt. Der Zugang zum Relay wird über ein Proof-of-Work-gestütztes Rendezvous und tokenbasierte Ratenbegrenzung geschützt. Der Prototyp ist in Rust auf Basis von QUIC umgesetzt und wird in einem Docker-basierten Testbett unter emulierten Netzbedingungen (`tc netem`) auf seine Roundtrip-Latenz hin evaluiert.

Abstract

This thesis designs and implements a *zero-knowledge network tunnel* for censorship-resistant communication: a provider-blind relay that forwards only ciphertext, while end-to-end encryption between client and origin is provided by the Noise protocol. Access to the relay is guarded by a proof-of-work rendezvous and per-token rate limiting. The prototype is implemented in Rust on top of QUIC and evaluated for round-trip latency in a Docker-based testbed under emulated network conditions (`tc netem`).

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation und Problemstellung	1
1.2	Zielsetzung	1
1.3	Forschungsfragen	2
1.4	Aufbau der Arbeit	2
2	Grundlagen	3
2.1	Providerblinde Relays und das Zero-Knowledge-Prinzip	3
2.2	QUIC als Transport	3
2.3	Das Noise-Protokoll	4
2.4	Proof-of-Work als Zugangsschutz	4
3	Architektur	5
3.1	Komponenten und Topologie	5
3.2	Schlüsselflüsse	5
3.3	Rollen-Dispatch am Edge	6
3.4	Entwurfsentscheidungen	7
4	Implementierung	8
4.1	Aufbau des Workspace	8
4.2	Kryptographische Bausteine (<code>ct-common</code>)	8
4.3	Rollen-Dispatch am Edge (<code>ct-edge</code>)	9
4.4	Agent und Client	10
5	Evaluation	11
5.1	Testbett und Methodik	11
5.2	Ergebnisse	11
5.3	Diskussion	11
5.4	Limitierungen	13
6	Fazit und Ausblick	14
6.1	Zusammenfassung	14
6.2	Ausblick	14
	Literatur	16

1 Einleitung

1.1 Motivation und Problemstellung

Dienste hinter NAT oder Firewalls für entfernte Nutzer erreichbar zu machen, ist ein alltäglicher Bedarf; die verbreiteten Tunneldienste lösen ihn jedoch um den Preis des Vertrauens: Der Verkehr wird beim Anbieter terminiert, sodass dieser den Klartext einsehen und die Verbindung auf Zuruf kappen kann. Für Nutzer mit einem Schutzbedarf gegen Zensur ist ein Anbieter, der den Datenverkehr *sehen* oder *unterbrechen* kann, jedoch selbst ein Angriffspunkt.

Diese Arbeit verfolgt den entgegengesetzten Entwurf: einen *providerblinden* (zero-knowledge) Tunnel, bei dem der Betreiber bewusst aus drei Dingen herausgehalten wird – dem *Inhalt* (die Nutzlast-Verschlüsselung terminiert erst am kundeneigenen Origin, nie beim Betreiber), dem *Vertrauen* (die Schlüssel sind kundenverankert, es gibt keine betreiberseitige PKI) und, soweit möglich, dem *Datenpfad* selbst. Die Zero-Knowledge-Eigenschaft ist dabei bewusst auf die *Nutzlast* begrenzt: Strukturelle Metadaten wie die Existenz eines Tunnels oder – im Relay-Fall – grobe Zeit- und Volumenzähler bleiben sichtbar. Durchsetzung reduziert sich auf einen einzigen, groben Hebel (Terminierung eines Tokens), der nur an einer eng definierten *rechtlichen Grenze* greift.

Aus dieser Haltung ergibt sich ein Spannungsfeld: Providerblindheit und Zensurresistenz erkaufte man sich mit zusätzlichen Vermittlungsschritten (Rendezvous, Relay) und kryptographischem Aufwand. Ob und in welchem Umfang diese Schutzziele die Latenz eines Tunnels belasten, ist die zentrale empirische Frage dieser Arbeit.

1.2 Zielsetzung

Ziel der Arbeit ist der Entwurf, die prototypische Implementierung und die emulationsbasierte Evaluation eines solchen Tunnels. Der Prototyp ist in Rust auf Basis von QUIC umgesetzt und realisiert den providerblinden Relay-Pfad Client → Edge → Agent → Origin mit einem Proof-of-Work-gestützten, ratenbegrenzten Rendezvous. Die Noise-basierte Ende-zu-Ende-Verschlüsselung ist als kryptographischer Baustein umgesetzt. Die Bewertung erfolgt in einem vollständig Docker-basierten Testbett, in dem Netzbedingungen (Verzögerung, Paketverlust, Bandbreite) mit `tc netem` emuliert und die Roundtrip-Latenz reproduzierbar vermessen werden.

1.3 Forschungsfragen

Die Arbeit gliedert sich an drei Forschungsfragen:

FF1 – Realisierbarkeit. Lässt sich ein providerblinder, zensurresistenter Tunnel aus etablierten Bausteinen – QUIC als Transport, Noise für die Ende-zu-Ende-Sicherheit, Proof-of-Work als Zugangsschutz – schlüssig entwerfen und implementieren?

FF2 – Latenz-Overhead. Welchen Grund-Overhead verursacht der providerblinde Vermittlungspfad (Verbindungsaufbau, PoW-Rendezvous, Mehrsprung-Relay) gegenüber einer direkten Verbindung?

FF3 – Verhalten unter Netzbedingungen. Wie skaliert die Tunnel-Roundtrip-Latenz mit Netzverzögerung und Paketverlust?

1.4 Aufbau der Arbeit

Kapitel 2 führt die technischen Grundlagen ein (providerblinde Relays, Noise, QUIC, Proof-of-Work). Kapitel 3 beschreibt den Systementwurf und dessen Entwurfsentscheidungen. Kapitel 4 dokumentiert die Umsetzung des Prototyps. Kapitel 5 stellt das Testbett und die Messergebnisse vor und beantwortet FF2 und FF3. Kapitel 6 fasst die Ergebnisse zusammen und gibt einen Ausblick auf die offenen Punkte.

2 Grundlagen

Dieses Kapitel führt die vier technischen Bausteine ein, auf denen der Entwurf aufsetzt: das Prinzip providerblinder Relays (Abschnitt 2.1), QUIC als Transport (Abschnitt 2.2), das Noise-Protokoll für die Ende-zu-Ende-Sicherheit (Abschnitt 2.3) und Proof-of-Work als Zugangsschutz (Abschnitt 2.4).

2.1 Providerblinde Relays und das Zero-Knowledge-Prinzip

Ein *Relay* vermittelt Verkehr zwischen zwei Endpunkten, ohne selbst Endpunkt zu sein. Klassische Tunneldienste terminieren die Verschlüsselung beim Betreiber; dieser kann den Klartext lesen und ist damit sowohl ein Vertrauensanker als auch ein Angriffs- und Druckpunkt. Ein *providerblindes* Relay kehrt diese Eigenschaft um: Es leitet ausschließlich Chiffretext weiter und besitzt keinen Schlüssel, mit dem sich die Nutzlast entschlüsseln ließe. Die Verschlüsselung terminiert erst am kundeneigenen Ziel (*Origin*).

Die Zero-Knowledge-Eigenschaft ist dabei präzise auf die *Nutzlast* begrenzt. Der Betreiber kann weder Inhalt lesen oder verändern noch das Origin-Schlüsselmaterial erlangen; sichtbar bleiben ihm hingegen strukturelle *Metadaten*: dass ein Tunnel existiert, die Koordinationsereignisse des Rendezvous sowie – nur im Relay-Fall – grobe Zeit- und Volumenzähler. Adressiert wird ein Tunnel nicht über einen Hostnamen, sondern über ein opakes *Routing-Token*, sodass dem Betreiber auch das Vermittlungsziel verborgen bleibt. Diese Trennung von blinder Nutzlast und sichtbaren Metadaten ist der konzeptionelle Kern der Arbeit.

2.2 QUIC als Transport

QUIC ist ein UDP-basiertes, gesichertes Transportprotokoll [2], dessen Sicherheitsschicht auf TLS 1.3 aufsetzt [4]. Für einen Tunnel ist es aus mehreren Gründen geeignet: Es multiplext mehrere Ströme über eine Verbindung ohne wechselseitiges Head-of-Line-Blocking unter Paketverlust, es unterstützt Verbindungsmigration bei wechselnden IP-Adressen und es erlaubt einen schnellen Verbindungsaufbau. Im hier verfolgten Entwurf baut der Agent die Verbindung *ausgehend* zum Edge auf, sodass auf der Kundenseite keine eingehenden Ports

geöffnet werden müssen; eine Client-Verbindung bildet sich auf einen QUIC-Strom ab. Wo ausgehendes UDP/443 blockiert ist, ist ein Rückfall auf HTTP/2 über TCP/443 vorgesehen. Die innere, providerblinde Sitzung (Abschnitt 2.3) läuft byteweise über diesen Transport und ist von der QUIC-eigenen TLS-Sicherung des Agent↔Edge-Abschnitts unabhängig.

2.3 Das Noise-Protokoll

Die Ende-zu-Ende-Sicherheit zwischen Client und Origin stellt das Noise-Protokoll-Framework [3] her – konkret ein Muster vom Typ `Noise_IK` mit statischen X25519-Schlüsselpaaren. Der Client kennt und *pinnt* den statischen öffentlichen Schlüssel des Origin (die *Origin Identity*); beide Seiten authentisieren sich über ihre statischen Schlüssel. Das Ergebnis ist ein gegenseitig authentisierter, vorwärtsgeheimer Kanal *ohne* Zertifizierungsstelle oder PKI. Gerade das Fehlen eines externen Ausstellers ist für die Zensurresistenz wesentlich: Es gibt keine Instanz, die zur Sperrung oder zum Ausstellen falscher Identitäten gedrängt werden könnte. Die Noise-Sitzung ist ein *innerer* Handshake über den QUIC-Strom und damit unabhängig von der Transportsicherung. Der Preis dieser Entkopplung ist, dass die Schlüsselverteilung in der Verantwortung des Kunden liegt: Clients müssen die Origin Identity außerhalb des Bandes erhalten, und der Schlüsseltausch (Rotation) ist eine explizite Operation ohne Ablauf-/Erneuerungsautomatik.

2.4 Proof-of-Work als Zugangsschutz

Da der Betreiber providerblind ist und bewusst auf WAF, Abuse-Feeds und Identitätsprüfung (KYC) verzichtet, fehlen ihm die üblichen Hebel gegen Denial-of-Service und Sybil-Angriffe. Als Ersatz dient ein gestaffeltes Schema, dessen Kern ein *Proof-of-Work* (PoW) ist [1]: Teure Operationen – eine Rendezvous-Anfrage oder das Belegen eines Relay-Platzes – erfordern den Nachweis einer verrichteten Rechenarbeit, typischerweise das Finden einer Nonce, deren Hash eine geforderte Schwierigkeit erfüllt. Ergänzt wird der Nachweis durch *Ratenbegrenzung* pro Token sowie durch Kapazitätsreserven. Der PoW ist zugleich der primäre Sybil-Schutz in Abwesenheit von KYC. Seine Grenze ist bekannt und wird in dieser Arbeit nicht ausgeblendet: Die Schwierigkeit muss Fluten abschrecken, ohne legitime Nutzung zu behindern, und ein hinreichend finanzierter Angreifer kann die Kosten schlicht bezahlen – missbrauchs- und abrechnungsseitige Sybil-Resistenz bleibt ein offener Punkt (Kapitel 6).

3 Architektur

Dieses Kapitel beschreibt den Systementwurf: die Komponenten und ihr Zusammenspiel (Abschnitt 3.1), die tragenden Flüsse (Abschnitt 3.2), das am Edge implementierte Rollen-Dispatch (Abschnitt 3.3) und die wesentlichen Entwurfsentscheidungen (Abschnitt 3.4).

3.1 Komponenten und Topologie

Der Entwurf kennt vier Datenpfad-Rollen und eine Steuerkomponente (Abbildung 3.1):

Origin. Der private Dienst, den der Kunde exponieren will. Hier terminiert die Verschlüsselung; das Origin-Schlüsselmaterial verlässt diesen Bereich nie.

Agent. Die kundenbetriebene, rein ausgehend verbindende Software. Sie ist Verwahrer des Origin-Schlüssels, prägt *Capabilities* und leitet Verkehr an den lokalen Origin weiter.

Edge. Der betreiberbetriebene, öffentlich erreichbare Knoten. Er koordiniert das Rendezvous und leitet – als Rückfall – ausschließlich Chiffretext weiter. Er kann nicht entschlüsseln und routet über ein opakes Routing-Token.

Client. Der Endpunkt, der einen Origin erreichen möchte. Er hält eine Capability und pinnt die Origin Identity.

Control Plane. Eine dünne, self-hosting-fähige Steuerkomponente für Enrollment, Tunnel-Registry und Rendezvous-Endpunkt. Sie hält weder Vertrauensmaterial noch Nutzlast.

3.2 Schlüsselflüsse

1. **Agent-Enrollment.** Ein einmaliges Join-Token bindet einen neu erzeugten Agent-Identitätsschlüssel an einen Mandanten; im Betrieb authentisiert sich der Agent über kurzlebigen mTLS.

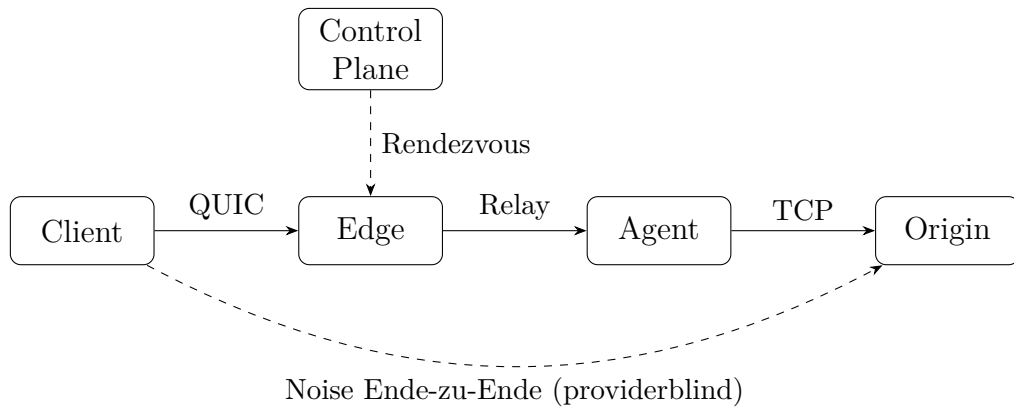


Abbildung 3.1: Komponenten und Datenpfad. Der Edge leitet nur Chiffretext weiter; die Noise-Sitzung ist Ende-zu-Ende zwischen Client und Origin und für den Edge undurchsichtig.

2. **Capability-Minting.** Der Agent erzeugt eine selbsttragende *Capability* – Routing-Token, Origin Identity und Edge-Adresse – die der Kunde außerhalb des Bandes an berechnete Clients verteilt. Besitz genügt, um einen Origin zu erreichen und zu authentisieren.
3. **Verbindungsaufbau.** Der Client legt seine Capability vor, durchläuft ein PoW- und ratenbegrenztes Rendezvous am Edge, wird über sein Routing-Token an den registrierten Agent geroutet und führt schließlich den Noise-Handshake zum Origin, gepinnt auf die Origin Identity.
4. **Revocation.** Widerruf erfolgt durch Rotation des Tokens und/oder des Origin-Schlüssels.

3.3 Rollen-Dispatch am Edge

Der Prototyp bündelt die Edge-Logik in einer einzigen Routine, die pro eingehender Verbindung anhand eines ersten Rollen-Bytes verzweigt:

Rolle 'A' (Agent). Der Agent meldet auf einem Kontrollstrom sein Routing-Token an; der Edge trägt die Tunnel-Verbindung in die Registry ein und quittiert. Das Token wird damit routbar.

Rolle 'C' (Client). Der Client durchläuft das PoW-Rendezvous, präsentiert sein Routing-Token, woraufhin der Edge die passende Agent-Verbindung nachschlägt, einen Strom zu ihr öffnet und beide Ströme byteweise koppelt (Relay).

Dieses Dispatch bildet die Trennung von Registrierung und Nutzung auf einem gemeinsamen QUIC-Transport ab und hält den Edge zustandsarm: Er kennt nur die Abbildung Token → Agent-Verbindung, nie den Klartext.

3.4 Entwurfsentscheidungen

Die Architektur folgt einigen bewusst getroffenen Entscheidungen, die in Architecture Decision Records festgehalten sind: ein providerblinder, Ende-zu-Ende-verschlüsselter Datenpfad; QUIC als Transport mit TCP-Rückfall; die Noise-basierte, PKI-freie Mesh-Authentisierung; ein einregionaler Edge um eine erststufige Tunnel-Registry; sowie eine dünne, self-hosting-fähige Control Plane, die weder Schlüssel noch Nutzlast sieht. Kapitel 4 zeigt, wie diese Entscheidungen in den Rust-Crates umgesetzt sind.

4 Implementierung

Dieses Kapitel dokumentiert den Prototyp: den Aufbau des Rust-Workspace (Abschnitt 4.1), die kryptographischen Bausteine (Abschnitt 4.2), das Rollen-Dispatch am Edge (Abschnitt 4.3) sowie Agent und Client (Abschnitt 4.4).

4.1 Aufbau des Workspace

Der Prototyp ist ein Rust-Cargo-Workspace aus fünf Crates (Tabelle 4.1). Der Datenpfad ist bewusst in Rust umgesetzt – speichersicher ohne Garbage-Collector, mit ausgereiften Bibliotheken für die benötigten Bausteine: **quinn** für QUIC, **rustls** (mit dem **ring**-Provider) für die Transportsicherung, **snow** für den Noise-Handshake, **ed25519-dalek** für Signaturen, **sha2** für den Proof-of-Work, **rcgen** für selbstsignierte Zertifikate und **tokio** als asynchrone Laufzeit.

4.2 Kryptographische Bausteine (**ct-common**)

ct-common bündelt die geteilten Typen. Eine **Capability** (Routing-Token, Origin Identity, Edge-Adresse) besitzt eine handgeschriebene binäre Kodierung mit definierter Fehlerbehandlung (**MalformedCapability**) statt einer generischen Serialisierung, um das Wire-Format kontrolliert zu halten. Der Proof-of-Work ist bewusst asymmetrisch: Das Prüfen kostet einen einzigen Hash, das Lösen wächst exponentiell mit der Schwierigkeit (Listing 4.1). Hinzu kommen der Noise-Handshake (Client- und Origin-Seite über **snow**), die signierten Credentials (**ed25519-dalek**) und eine Fenster-basierte Ratenbegrenzung pro Token.

```
// Verify: ein einziger Hash.
pub fn verify(challenge: &Challenge, solution: u64) -> bool {
    leading_zero_bits(&hash(&challenge.nonce, solution))
        >= challenge.difficulty as u32
}

// Solve: Brute force, Kosten ~2^difficulty.
pub fn solve(challenge: &Challenge) -> u64 {
    let mut solution = 0u64;
    loop {
        if verify(challenge, solution) { return solution; }
        solution += 1;
    }
}
```

Tabelle 4.1: Crates des Workspace und ihre Rolle.

Crate	Rolle
ct-common	Gemeinsame Typen: Capability, Credential, Noise, PoW, Ratenbegrenzung
ct-edge	Edge-Daemon: Registry, Rollen-Dispatch, Relay
ct-agent	Agent-Daemon: Tunnel-Registrierung, Weiterleitung zum Origin
ct-client	Client-Werkzeug: Rendezvous, Tunnel, Bench-Modus
ct-control-plane	Dünne Steuerkomponente: Enrollment, Registry, Ausstellung

```
}
}
```

Listing 4.1: Asymmetrischer Proof-of-Work: Prüfen ist billig, Lösen teuer (ct-common::pow).

4.3 Rollen-Dispatch am Edge (ct-edge)

Der Kern des Edge ist die Routine `serve_connection` (Listing 4.2): Sie liest ein erstes Rollen-Byte und verzweigt. Ein Agent ('A') meldet sein Token an und wird in die Registry (`EdgeState`) eingetragen; ein Client ('C') erhält die PoW-Challenge, sein Request wird geprüft, das Token zur Agent-Verbindung aufgelöst und beide QUIC-Ströme werden byteweise gekoppelt. Der Edge hält damit nur die Abbildung Token → Verbindung und sieht nie den Klartext.

```
match role[0] {
  b'A' => {
    // Agent: Tunnel registrieren
    recv.read_exact(&mut token).await?;
    state.register(RoutingToken(token), conn.clone());
    send.write_all(b"OK").await?;
  }
  b'C' => {
    // Client: PoW-Rendezvous, dann Relay
    send.write_all(&challenge_bytes).await?;
    recv.read_exact(&mut req).await?;
    let token = check_request(challenge, &req)?; // PoW prüfen
    let agent = state.route(&token).ok_or("no agent")?;
    let (a_send, a_recv) = agent.open_bi().await?;
    relay_quic(send, recv, a_send, a_recv).await?; // koppeln
  }
  other => return Err(format!("unknown role: {other}").into()),
}
```

Listing 4.2: Rollen-Dispatch am Edge (gekürzt, ct-edge::serve).

4.4 Agent und Client

Der Agent (`ct-agent`) wählt ausgehend zum Edge, registriert seinen Tunnel (`'A'`) und überbrückt anschließend jeden vom Edge weitergeleiteten QUIC-Strom mittels `copy_bidirectional` auf eine TCP-Verbindung zum lokalen Origin. Der Client (`ct-client`) wählt den Edge, durchläuft das PoW-Rendezvous (`'C'`), tunnelt seine Nutzlast und verifiziert die Antwort; ein zusätzlicher Bench-Modus misst die Roundtrip-Latenz über mehrere Iterationen und dient als Datenquelle der Evaluation (Kapitel 5). Alle drei Binaries beziehen ihre Konfiguration aus Umgebungsvariablen und koordinieren sich im Testbett über ein geteiltes Volume (Edge-Zertifikat, Capability).

Die Umsetzung ist testgetrieben entstanden: Der Workspace umfasst 94 automatisierte Tests – von den reinen Kodierungs- und PoW-Funktionen bis zu Integrationstests, die einen vollständigen Client→Edge→Agent-Roundtrip über echtes QUIC ausführen.

5 Evaluation

Dieses Kapitel wertet die Roundtrip-Latenz des Tunnels im Docker-Testbett unter emulierten Netzbedingungen aus und beantwortet damit die Forschungsfragen FF2 (Latenz-Overhead) und FF3 (Verhalten unter Netzbedingungen).

5.1 Testbett und Methodik

Das Testbett ist vollständig containerisiert (Docker Compose): Edge, Agent, Origin (ein TCP-Echo-Dienst) und Client laufen auf einem statischen Netz. Die Link-Impairment wird mit `tc netem` auf der Egress-Schnittstelle des Edge gesetzt und ist über Umgebungsvariablen (Verzögerung, Verlust, Bandbreite) parametrisiert. Der Client misst im Bench-Modus je Bedingung $n = 5$ vollständige Roundtrips – Verbindungsaufbau, PoW-Rendezvous, Tunnel und Echo-Verifikation – jeweils mit *frisch aufgebauter* QUIC-Verbindung.

Die Auswertung ist vollständig reproduzierbar und ihrerseits containerisiert: `scripts/sweep.sh` fährt die Bedingungsmatrix ab und sammelt die Ergebnisse als CSV, `scripts/plot.sh` erzeugt die Abbildungen (Matplotlib in einem Python-Container) und `scripts/tabulate.py` die Tabelle. Vermessen wurde die Matrix Verzögerung $\in \{0, 20, 50\}$ ms $\times$ Verlust $\in \{0, 2\}$ %.

5.2 Ergebnisse

Tabelle 5.1 fasst alle Messpunkte zusammen; Abbildung 5.1 zeigt die Latenz über der Verzögerung, Abbildung 5.2 über dem Paketverlust.

5.3 Diskussion

FF2 – Latenz-Overhead. Ohne Netzverzögerung beträgt der Roundtrip im Mittel 6,8 ms; dies ist der Eigenanteil des Tunnels aus QUIC-Handshake, PoW-Rendezvous und Mehrsprung-Relay. Über die Messpunkte (0 ms $\rightarrow$ 6,8; 20 ms $\rightarrow$ 92,7; 50 ms $\rightarrow$ 217,9) skaliert die Latenz näherungsweise linear: $RT \approx 6,8 + 4,2 \cdot d$ mit d = einseitige Edge-Verzögerung in Millisekunden. Der Faktor $\approx 4,2$ spiegelt wider, dass jeder Anwendungs-Roundtrip die verzögerte Egress mehrfach quert (Verbindungsaufbau, Rendezvous und der Relay-Pfad Client $\rightarrow$ Edge $\rightarrow$ Agent $\rightarrow$ Origin).

Tabelle 5.1: Tunnel-Roundtrip-Latenz unter emulierten Netzbedingungen (`tcnetem`).

Verzögerung	Verlust	n	Mittel (ms)	p50 (ms)	p95 (ms)	Min (ms)	Max (ms)
0ms	0%	5	6.8	6.6	7.9	5.8	7.9
0ms	2%	5	6.9	6.9	8.2	5.5	8.2
20ms	0%	5	92.7	88.7	109.2	88.1	109.2
20ms	2%	5	92.2	88.2	108.5	88.0	108.5
50ms	0%	5	217.9	207.5	260.4	205.5	260.4
50ms	2%	5	237.5	207.7	306.5	207.5	306.5

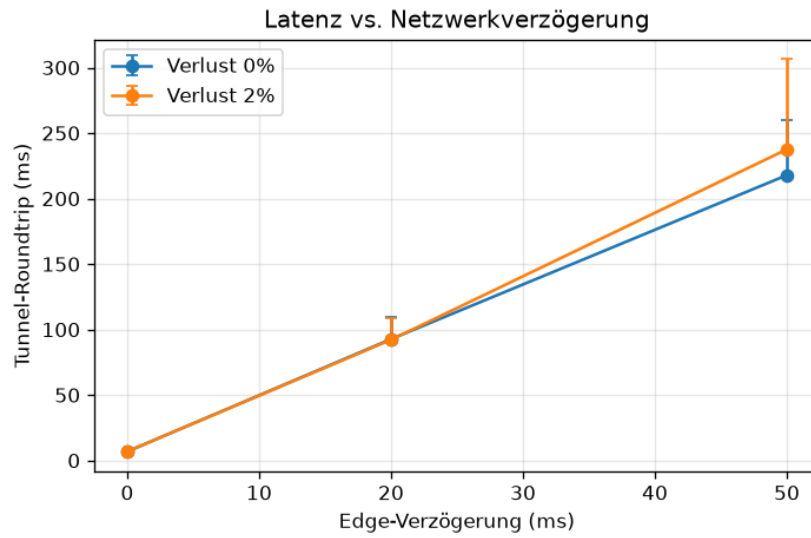


Abbildung 5.1: Tunnel-Roundtrip über der Edge-Verzögerung, je Verlustwert eine Serie; der obere Fehlerbalken markiert das p95.

und zurück). Da pro Iteration frisch verbunden wird, trägt der Handshake einen festen Vielfachen der Umlaufzeit bei.

FF3 – Verhalten unter Paketverlust. Bei niedriger Umlaufzeit (0 ms, 20 ms) ist ein Verlust von 2 % im Mittel nicht messbar (6,8→6,9 ms bzw. 92,7→92,2 ms, innerhalb der Streuung). Bei 50 ms hingegen steigt der Mittelwert um rund 9 % (217,9→237,5 ms) und das p95 um rund 18 % (260,4→306,5 ms), während der Median nahezu unverändert bleibt (p50 $\approx$ 207 ms). Verlustbedingte Neuübertragungen wirken sich also proportional zur Umlaufzeit aus und zeigen sich primär im Tail (p95), nicht im Median.

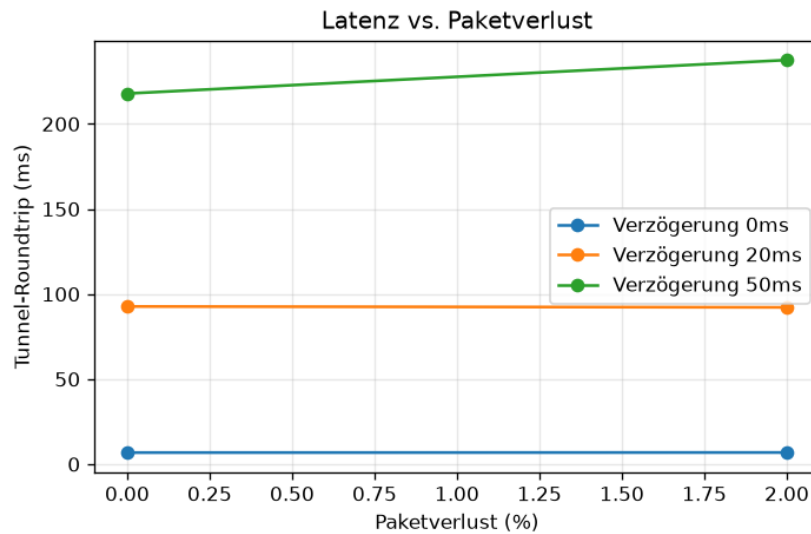


Abbildung 5.2: Tunnel-Roundtrip über dem Paketverlust, je Verzögerungswert eine Serie.

5.4 Limitierungen

Die Ergebnisse sind unter drei Einschränkungen zu lesen. Erstens wirkt die Impairment nur auf der Edge-Egress und damit nicht symmetrisch; die absolute Latenz ist eine untere Schranke für eine voll bidirektional verzögerte Strecke. Zweitens dient $n = 5$ hier der Demonstration der reproduzierbaren Mess-Pipeline, nicht einer statistisch belastbaren Aussage – größere Stichproben und eine breitere Matrix (inklusive Bandbreitenbegrenzung) sind über dieselben Skripte unmittelbar zugänglich. Drittens überzeichnet der Frisch-Verbindungsaufbau je Iteration den Handshake-Anteil; ein Keep-Alive-Datenpfad läge niedriger. Diese Punkte greift der Ausblick (Kapitel 6) auf.

6 Fazit und Ausblick

6.1 Zusammenfassung

Diese Arbeit hat einen providerblinden, zensurresistenten Netzwerktunnel entworfen, prototypisch in Rust auf Basis von QUIC implementiert und in einem vollständig containerisierten Testbett unter emulierten Netzbedingungen evaluiert. Gemessen an den drei Forschungsfragen ergibt sich folgendes Bild.

FF1 – Realisierbarkeit. Ein providerblinder Tunnel lässt sich aus etablierten Bausteinen schlüssig zusammensetzen. Der Prototyp realisiert den providerblinden Vermittlungspfad Client→Edge→Agent→Origin mit einem PoW-gestützten, ratenbegrenzten Rendezvous über einen einheitlichen QUIC-Transport mit Rollen-Dispatch am Edge; die kryptographischen Bausteine – Noise-Handshake, signierte Credentials, Proof-of-Work, Ratenbegrenzung – sind in `ct-common` umgesetzt und durch 94 automatisierte Tests, bis hin zu End-to-End-Roundtrips über echtes QUIC, abgesichert. Die Realisierbarkeit ist damit für Architektur und Kernmechanismen belegt; als offener Integrationsschritt verbleibt, die Noise-Ende-zu-Ende-Sitzung auf dem Live-Datenpfad zu verankern (Abschnitt 6.2).

FF2 und FF3 – Latenz. Der Eigen-Overhead des Tunnels ist im unbelasteten Fall gering (Mittel 6,8 ms) und skaliert linear mit der Netzverzögerung ($RT \approx 6,8 + 4,2 \cdot d$); der Faktor erklärt sich aus dem mehrfachen Queren der verzögerten Strecke pro Roundtrip. Paketverlust von 2 % wirkt sich erst bei großer Umlaufzeit spürbar aus und dort primär im Tail (p95 bei 50 ms um rund 18 % erhöht), nicht im Median. Die Schutzziele Providerblindheit und Zensurresistenz sind also mit einem berechenbaren, im Wesentlichen von der Netzstrecke dominierten Latenzaufschlag vereinbar.

6.2 Ausblick

Prototyp. Der nächste Schritt ist die Verankerung der Noise-Ende-zu-Ende-Sitzung auf dem Live-Pfad sowie der *direkte* Peer-to-Peer-Datenpfad mit NAT-Hole-Punching und Edge-Relay nur als Rückfall – im aktuellen Prototyp läuft

der Verkehr stets über das Relay. Hinzu kommen der HTTP/2-über-TCP-Rückfall für blockiertes UDP sowie eine belastbarere Evaluation: größere Stichproben, symmetrische und bidirektionale Impairment, Bandbreitenbegrenzung und ein Keep-Alive-Datenpfad neben dem Frisch-Verbindungsaufbau.

Offene Risiken. Unabhängig von der Technik bleiben drei Fragen offen, die über die Betreibbarkeit der Zensurresistenz-Haltung entscheiden: die abrechnungsseitige Sybil-/Missbrauchsresistenz (Proof-of-Work deckt Fluten, aber keinen finanzierten Angreifer ab), die Wahl von Jurisdiktion, Hosting und Zahlungswegen sowie ein konkreter operativer Prozess für die einzige Durchsetzungsgrundlage an der rechtlichen Grenze.

Zurückgestellte Funktionen. Über die v1 hinaus vorgesehen sind die Browser-Plane (anonymer Browserzugang per SNI-Passthrough), vollständige Selbst-Hostbarkeit und dezentrale Koordination, Agent-Redundanz sowie komfortable Abläufe für die Schlüsselrotation. Diese Punkte sind im Backlog des Projekts festgehalten und bewusst von der v1 getrennt, um den Kern – einen belegbar providerblinden Tunnel – zuerst tragfähig zu machen.

Literatur

- [1] Adam Back. “Hashcash – A Denial of Service Counter-Measure”. In: Technical report. 2002.
- [2] Jana Iyengar und Martin Thomson. *QUIC: A UDP-Based Multiplexed and Secure Transport*. RFC 9000. Internet Engineering Task Force (IETF), 2021. DOI: 10.17487/RFC9000.
- [3] Trevor Perrin. *The Noise Protocol Framework*. <https://noiseprotocol.org/noise.html>. Revision 34. 2018.
- [4] Martin Thomson und Sean Turner. *Using TLS to Secure QUIC*. RFC 9001. Internet Engineering Task Force (IETF), 2021. DOI: 10.17487/RFC9001.